

Publishing Doxygen and Sphinx with Nix and Rake

Rohit Goswami · 2020-09-22 · documentation · workflow · nix · cpp · ruby · rake

Automating documentation deployment with Travis, `rake` and `nix`

Background

In the [previous post](#) we generated documentation using Doxygen with Exhale to handle Sphinx. Now we will clean up the earlier workflow with `rake` and ensure the environment is reproducible with `nix` while deploying to [Travis CI](#).

Setup

A quick reminder of the setup we generated in the last post:

```
tree -d $prj/ -L 2
```

```
.
├── docs
│   ├── Doxygen
│   └── Sphinx
├── nix
│   └── pkgs
├── projects
│   └── symengine
└── scripts
```

8 directories

We had further setup files to enable documentation generation with a manual two stage process (handling `doxygen` and `sphinx` separately).

```
cd docs/Doxygen
doxygen Doxyfile-prj.cfg
cd ../Sphinx
make html
mv build/html ../../public
```

This might be extracted into a simple `build.sh` script, and then we might decide to have a `clean.sh` script and then we might try to replicate all the functionality of a good build system with scripts.

Thankfully, we will instead start with a `build` script defined as above to transition to `nix`, before using an actual build tool for our dirty work.

Adding Nix

It wouldn't make sense for me to not stick `nix` into this. I recall the dark days of setting up `Dockerfiles` to ensure reproducible environments on Travis.

At this point one might assume we will leverage the `requirements.txt` based workflow described earlier in [Niv](#) and [Mach-Nix for Nix Python](#). While this would make sense, there are two barriers to its usage:

- It is slower than a `poetry` build, as dependency resolution is performed
- It does not play well with existing projects
 - Most `python` projects do not rely solely on `requirements.txt` `[^fn:1]`

Poetry2Nix

Recall that as `sphinx` is originally meant for and most often used for Python projects, we will need to consider the possibility (remote though it is) that there might be users who would like to test the documentation without setting up `nix`.

Thus we will look to the [poetry2nix project](#) instead. We note the following:

- The `poetry2nix` setup is faster (as it consumes a `lockfile` instead of solving dependencies from `requirements.txt`)
 - `mach-nix` however, is more flexible and can make use of the `poetry2nix` overrides
- In a strange chicken and egg problem, we will have to manually generate the lockfile, thereby creating an impure `poetry` project for every update, though the `nix` setup will not need it later
 - This is one of the major reasons to prefer `mach-nix` for newer projects

Shell Environment

We prep our sources in the usual way, by running `niv init` in the project root to generate the `nix/` folder and the sources therein. With all that in mind, the `shell.nix` file at this point is fairly standard, keeping the general `niv` setup in mind (described in a [previous Nix tutorial](#)):

```
# -*- mode: nix-mode -*-
let
  sources = import ./nix/sources.nix;
  pkgs = import sources.nixpkgs { };
  customPython = pkgs.poetry2nix.mkPoetryEnv { projectDir = ./.; };
in pkgs.mkShell {
  buildInputs = with pkgs; [ doxygen customPython rake darkhttpd ];
}
```

Where the most interesting aspect is that the `projectDir` is to be the location of the project root, though both `poetrylock` and `pyproject` variables are supported.

Refactoring

We consider the problem of refactoring the `build.sh` script:

```
#!/usr/bin/env bash
cd docs/Doxygen
doxygen Doxyfile-prj.cfg
cd ../Sphinx
make html
mv build/html../../public
```

Without resorting to methods such as `nix-shell --run build.sh --pure`.

Nix Bash

Script in hand, we would like to be able to run it directly in the `nix` environment. We modify the script as follows:

```
#!/usr/bin/env nix-shell
#! nix-shell deps.nix -i bash

# Build Doxygen
cd docs/Doxygen
doxygen Doxyfile-syme.cfg

# Build Sphinx
cd ../Sphinx
make html
mv build/html ../../public

# Local Variables:
# mode: shell-script
# End:
```

This calls on a `deps.nix` [^fn:2] which we shall generate in a manner very reminiscent of the `shell.nix` [^fn:3] as follows:

```
let
  sources = import ../../nix/sources.nix;
  pkgs = import sources.nixpkgs { };
  customPython = pkgs.poetry2nix.mkPoetryEnv { projectDir = ../../.; };
in pkgs.runCommand "dummy" {
  buildInputs = with pkgs; [ doxygen customPython ];
} ""
```

Only the paths have changed, and instead of creating and returning a shell environment with `mkShell` we instead “run” a derivation instead. At this point we can run this simply as:

```
./scripts/build.sh
```

This is reasonably ready (as a first draft) for being incorporated into a continuous integration workflow.

Travis CI

Seeing as Travis provides first class `nix` support, as well as excellent integration with GitHub, we will prefer it.

Settings

A minor but necessary evil is setting up a PAP (personal access token) from [here](#). Depending on what repositories are being used, the scope should encompass `repo` permissions (minimally `public_repo`), and `admin:org` permissions might be required.

Having obtained the token, we will need to go to the Settings section on the Travis web-UI and add the token as an environment variable, we might be partial to a name like `GH_TOKEN`.

Environment Variables

Customize your build using environment variables. For secure tips on generating private keys [read our documentation](#)

GH_TOKEN



Available to all branches



Figure 1: Settings at `travis-ci.com/host/proj/settings`

Build Configuration

We will leverage the following configuration:

```
language: nix

before_install:
  - sudo mkdir -p /etc/nix
  - echo "substituters = https://cache.nixos.org/ file://$HOME/nix.store" | sudo tee -a /etc/nix/nix.conf > /dev/null
  - echo 'require-sigs = false' | sudo tee -a /etc/nix/nix.conf > /dev/null

before_script:
  - sudo mkdir -p /etc/nix && echo 'sandbox = true' | sudo tee /etc/nix/nix.conf

script:
  - scripts/build.sh

before_cache:
  - mkdir -p $HOME/nix.store
  - nix copy --to file://$HOME/nix.store -f shell.nix buildInputs

cache:
  nix: true
  directories:
    - $HOME/nix.store

deploy:
  provider: pages
  local_dir: ./public/
  skip_cleanup: true
  github_token: $GH_TOKEN # Set in the settings page of your repository, as a secure variable
  keep_history: true
  target_branch: master # Required for user pages
  on:
    branch: src
```

Where all the action is essentially in `script` and `deploy`. Note however, that the `before_cache` step should change if there is a `default.nix` instead. We will in this case, consider the situation of having an organization or user page being the deploy target.

Rake

Usable though the preceding setting is, it is still rather unwieldy in that:

- there are a bunch of artifacts which need to be cleaned manually
- it is fragile and tied to the folder names

We can fix this with any of the popular build systems, however here we will focus on the excellent `rake` [^fn:4]. We shall commit to our course of action by removing `make`.

```
cd docs/Sphinx
rm Makefile make.bat # other make cruft
```

Components

Variables

We will begin by requiring `rake` and setting basic variables.

```
require 'rake'

CWD = File.expand_path(__dir__)
DOXYFILE = "Doxyfile-prj.cfg"
OUTDIR = File.join(CWD, "public")
SPHINXDIR = File.join(CWD, "docs/Sphinx")
```

This section should give a fairly clear idea of how the `Rakefile` itself is essentially pure `ruby` code. We are now beginning to have more complete control of how our project is structured.

Tasks

The general form of a `task` is simply:

```
desc "Blah blah"
task :name do
  # Something
end
```

Some variations of this will be considered when appropriate.

Clean

A `clean` task is a good first task, being as it is almost trivial in all build systems.

```
desc "Clean the generated content"
task :clean do
  rm_rf "public"
  rm_rf "docs/Doxygen/gen_docs"
  rm_rf "docs/Sphinx/build"
end
```

Serve

We will use the [lightweight darkhttpd server](#) for our generated documentation.

```
desc "Serve site with darkhttpd"
task :darkServe, [:port] do |task, args|
  args.with_defaults(:port => "1337")
  sh "darkhttpd #{OUTDIR} --port #{args.port}"
end
```

Note that we have leveraged the `args` system in this case, and also used the top-level `OUTDIR` variable.

Doxygen

Since the `doxygen` output is a pre-requisite, it makes sense to set it up early on.

```
desc "Build doxygen"
task :mkDoxy do
  Dir.chdir(to = File.join(CWD, "docs/Doxygen"))
  system('doxygen', DOXYFILE)
end
```

Sphinx

This task will depend on having the `doxygen` output, so we will express this idiomatically by making the `doxygen` task run early on.

```
desc "Build Sphinx"
task :mkSphinx, [:builder] => ["mkDoxy"] do |task, args|
  args.with_defaults(:builder => "html")
  Dir.chdir(to = File.join(CWD, "docs/Sphinx"))
  sh "poetry install"
  sh "poetry run sphinx-build source #{OUTDIR} -b #{args.builder}"
end
```

There are some subtleties here, in particular:

- The task is meant to run **without** `nix`
- We use the `args` setup as before

No Nix Meta

With this we can now set up a task to build the documentation without having `nix`.

```
desc "Build site without Nix"
task :noNixBuild => "mkSphinx" do
  Rake::Task["darkServe"].execute
end
```

The main take-away here is that we finally call the `Rake` library itself, but within the task, which means the dependency tree is respected and we get `doxygen->sphinx->darkhttpd` as required.

Nix Builder

For `nix` use, we are unable to enter the `nix` environment from within the `Rakefile` itself. We work around this by being more descriptive.

```
desc "Build Nix Sphinx, use as nix-shell --run 'rake mkNixDoc' --pure"
task :mkNixDoc, [:builder] => "mkDoxy" do |task, args|
  args.with_defaults(:builder => "html")
  Dir.chdir(to = SPHINXDIR)
  sh "sphinx-build source #{OUTDIR} -b #{args.builder}"
end
```

Final Form

The final `Rakefile` shall be (with a default task defined):

```

require 'rake'

# Variables
CWD = File.expand_path(__dir__)
DOXYFILE = "Doxyfile-prj.cfg"
OUTDIR = File.join(CWD, "public")
SPHINXDIR = File.join(CWD, "docs/Sphinx")

# Tasks
task :default => :darkServe

desc "Clean the generated content"
task :clean do
  rm_rf "public"
  rm_rf "docs/Doxygen/gen_docs"
  rm_rf "docs/Sphinx/build"
end

desc "Serve site with darkhttpd"
task :darkServe, [:port] do |task, args|
  args.with_defaults(:port => "1337")
  sh "darkhttpd #{OUTDIR} --port #{args.port}"
end

desc "Build Nix Sphinx, use as nix-shell --run 'rake mkNixDoc' --pure"
task :mkNixDoc, [:builder] => "mkDoxy" do |task, args|
  args.with_defaults(:builder => "html")
  Dir.chdir(to = SPHINXDIR)
  sh "sphinx-build source #{OUTDIR} -b #{args.builder}"
end

desc "Build site without Nix"
task :noNixBuild => "mkSphinx" do
  Rake::Task["darkServe"].execute
end

desc "Build doxygen"
task :mkDoxy do
  Dir.chdir(to = File.join(CWD, "docs/Doxygen"))
  system('doxygen', DOXYFILE)
end

desc "Build Sphinx"
task :mkSphinx, [:builder] => ["mkDoxyRest"] do |task, args|
  args.with_defaults(:builder => "html")
  Dir.chdir(to = File.join(CWD, "docs/Sphinx"))
  sh "poetry install"
  sh "poetry run sphinx-build source #{OUTDIR} -b #{args.builder}"
end

```

Travis

We are now in a position to fix our `travis` build configuration. Simply replace the old and fragile `build.sh` script section with the following:

```

script:
  - nix-shell --run "rake mkNixDoc" --show-trace --verbose --pure

```

Direnv

As a bonus section, consider the addition of the following `.envrc` for those who keep multiple `ruby` versions:

```
eval "$(rbenv init -)"  
rbenv shell 2.6.2  
rake -T
```

Activate this with the usual `direnv allow`. This has the added benefit of listing the defined tasks when `cd` 'ing into the project directory.

Conclusions

A lot has happened on the tooling end, even though the documentation itself has not been updated further. We have managed to setup a solid environment which is both reproducible and also amenable to users who do not have `nix`. We have also setup a build system, which can help us in many more ways as well (asset optimization through the `rails` pipeline). In the next post, we will return to the documentation itself for further tinkering.